

```
manchodu@Srini:~/OSLAB$ nano m7.c
manchodu@Srini:~/OSLAB$ gcc -o m7 m7.c
manchodu@Srini:~/OSLAB$ ./m7
=====
      RACE CONDITION PROGRAM
=====
Number of threads : 4
Iterations/thread: 1000000

Expected counter value : 4000000
Actual counter value   : 1616853
Race condition detected!
manchodu@Srini:~/OSLAB$ |
```

```

=====
                RACE CONDITION PROGRAM
=====
Number of threads : 4
Iterations/thread: 1000000

Expected counter value : 4000000
Actual counter value   : 1616853
Race condition detected!
manchodu@Srini:~/OSLAB$ cat m7.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

long long counter = 0;

void *increment_counter(void *arg)
{
    int i;

    for (i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int i;

    printf("=====\n");
    printf("                RACE CONDITION PROGRAM\n");
    printf("=====\n");

    printf("Number of threads : %d\n", NUM_THREADS);
    printf("Iterations/thread: %d\n", ITERATIONS);

    for (i = 0; i < NUM_THREADS; i++)
    {

```

```

int main()
{
    pthread_t threads[NUM_THREADS];
    int i;

    printf("=====\n");
    printf("      RACE CONDITION PROGRAM\n");
    printf("=====\n");

    printf("Number of threads : %d\n", NUM_THREADS);
    printf("Iterations/thread: %d\n", ITERATIONS);

    for (i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                          increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }

    for (i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("\nExpected counter value : %lld\n",
          (long long)NUM_THREADS * ITERATIONS);

    printf("Actual counter value   : %lld\n",
          counter);

    if (counter != (long long)NUM_THREADS * ITERATIONS)
        printf("Race condition detected!\n");
    else
        printf("Race condition was not visible in this run.\n");

    return 0;
}

```

```
manchodu@Srini:~/OSLAB$ cat m8.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

long long counter = 0;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *increment_counter(void *arg)
{
    int i;

    for (i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&mutex);

        counter++;

        pthread_mutex_unlock(&mutex);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int i;

    printf("=====\n");
    printf("      MUTEX SYNCHRONIZATION PROGRAM\n");
    printf("=====\n");

    printf("Number of threads : %d\n", NUM_THREADS);
    printf("Iterations/thread: %d\n", ITERATIONS);

    for (i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                          increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }
}
```

```

int main()
{
    pthread_t threads[NUM_THREADS];
    int i;

    printf("=====\n");
    printf("      MUTEX SYNCHRONIZATION PROGRAM\n");
    printf("=====\n");

    printf("Number of threads : %d\n", NUM_THREADS);
    printf("Iterations/thread: %d\n", ITERATIONS);

    for (i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                          increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }

    for (i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("\nExpected counter value : %lld\n",
          (long long)NUM_THREADS * ITERATIONS);

    printf("Actual counter value   : %lld\n",
          counter);

    if (counter == (long long)NUM_THREADS * ITERATIONS)
        printf("Counter value is correct.\n");
    else
        printf("Unexpected counter value.\n");

    pthread_mutex_destroy(&mutex);

    return 0;
}

```